

Supplementary Material

Are Archived NBA Consensus Moneylines Calibrated? A Pre-Specified Test of Favorite Pricing Across 15,351 Games

Jacob Burdier · The Scholars' Academy · August 2026

Everything here is generated by the replication materials at <https://github.com/jacobburdier05/nba-moneyline-calibration> and can be regenerated with a single command. Section references are to the main paper.

S1. Shin's normalization equals the additive rule for a two-outcome book

Section 5.4 of the main paper reports that Shin (1993) and the additive rule give identical results. Because that claim is unusual and changes how Table 5 should be read, the derivation is written out here rather than simply stated. Every step is checked against the sample by `src/shin_equivalence_proof.py`.

Let q_1 and q_2 be the raw implied probabilities of the two sides and $P = q_1 + q_2$ the booksum, with P greater than one. Shin sets

$$pi_i = [\text{sqrt}(z^2 + 4(1 - z) q_i^2 / P) - z] / (2(1 - z))$$

with z chosen so the pair sums to one. The additive rule sets $pi_i = q_i - (P - 1)/2$.

S1.1 Invert for z

Rearrange to $2 pi (1 - z) + z = \text{sqrt}(z^2 + 4(1 - z) q^2 / P)$ and square both sides. The z -squared terms cancel, and dividing through by $4(1 - z)$, nonzero for z below one, leaves

$$pi^2 + z pi (1 - pi) = q^2 / P, \quad \text{so} \quad z = (q^2 / P - pi^2) / (pi (1 - pi))$$

S1.2 Require both sides to return the same z

Shin's z is a single property of the book, so the expression must give the same value for each side. Because the pair sums to one, $1 - pi_1$ equals pi_2 and $1 - pi_2$ equals pi_1 , so both denominators equal $pi_1 pi_2$ and the requirement reduces to equal numerators:

$$(q_1^2 - q_2^2) / P = pi_1^2 - pi_2^2, \quad \text{that is} \quad (q_1 - q_2)(q_1 + q_2)/P = (pi_1 - pi_2)(pi_1 + pi_2)$$

Since $q_1 + q_2 = P$ by definition and $pi_1 + pi_2 = 1$ by the constraint, this collapses to $q_1 - q_2 = pi_1 - pi_2$. Shin's rule preserves the difference of the raw probabilities.

S1.3 Solve

Two linear conditions now pin the pair down uniquely: the sum is one and the difference is $q_1 - q_2$. Therefore $\pi_1 = (1 + q_1 - q_2)/2$. The additive rule gives $q_1 - (q_1 + q_2 - 1)/2$, which is the same expression. The two rules coincide.

S1.4 Numerical verification

Check	Maximum absolute error
S1.2, difference preserved	3.3e-16
S1.3, closed form $\pi_1 = (1 + q_1 - q_2)/2$	2.8e-16
Equivalence, $ \text{Shin} - \text{additive} $	2.8e-16
Sum constraint, $ \pi_1 + \pi_2 - 1 $	2.2e-16
Recovered z consistent across both sides	1.5e-14

Table S1. Verified across the 15,350 games with a positive margin. All errors are at the limit of double precision.

The recovered insider share z ranges from 0.0037 to 0.0718 with a mean of 0.0384, so these are real solutions well inside the model's valid range rather than edge cases where it breaks down. One game in the archive is quoted with a booksum below one, where Shin's model has no valid solution; it is left out here and falls back to proportional in Table 5 of the main paper.

S1.5 Scope, and an implementation note

Matching answers do not mean the two rules express the same idea. Shin's comes from a model of a bookmaker pricing against insider money, and is meant to load margin onto longshots. The additive rule is a mechanical convention with no story about behavior behind it. They agree only because a two-outcome market leaves a single degree of freedom once the probabilities are required to sum to one and their difference is held fixed. With three or more outcomes they part ways.

One coding note, recorded because it produced wrong numbers before it was caught. Computed directly, the numerator $\sqrt{z^2 + a} - z$ subtracts two nearly equal quantities as z approaches one. In double precision it loses most of its accurate digits, which sends the root finder to values whose probabilities sum to roughly 0.98 rather than one. Multiplying by the conjugate gives the algebraically identical but numerically stable form $\pi = 2 q^2 / [P (\sqrt{z^2 + a} + z)]$, which is what the code uses.

S2. Exact power, and the normal approximation

The minimum detectable effect quoted in the main paper is the standard two-sided normal approximation, not an exact inversion of the Poisson-binomial power function. Because the two are not guaranteed to agree, `src/power_curve.py` computes the exact version. It builds the exact Poisson-binomial distribution by discrete Fourier transform of its characteristic function, using the conjugate symmetry so only half the frequencies are evaluated. It then finds the exact two-sided rejection region under the null and inverts exact power at 80 percent.

Quantity	Value
Exact rejection region	wins $\leq 5,425$ or $\geq 5,554$
Exact size (discreteness, not .05)	.0484
MDE, normal approximation	1.3281 pp
MDE, exact Poisson-binomial	1.3242 pp
Difference	0.0039 pp
Exact null mean vs analytic sum(p)	5489.4484 vs 5489.4484
Exact null SD vs sqrt(sum p(1-p))	32.4259 vs 32.4259

Table S2. The exact null mean and standard deviation match their analytic counterparts to four decimal places, which is an independent check on the primary test's variance.

True gap (pp)	Exact power	Normal approximation
0.50	17.75%	18.39%
1.00	55.56%	55.94%
1.33 (MDE)	80.35%	80.11%
1.50	88.97%	88.57%
2.00	99.00%	98.81%
3.01 (break-even)	>99.99%	>99.99%

Table S3. Exact against approximate power. The largest disagreement anywhere in the range is 0.64 percentage points of power.

For comparison, the illustrative narrow bucket in section 1 of the main paper, 74 games at an assumed win probability of .85, has 11.2 percent power against the same 3.01-point break-even margin.

S3. The withdrawn literature range

An earlier draft of this paper stated that favorite bias in the range commonly claimed, roughly 1.7 to 5.1 percentage points, had been reported in prior work, and

Figure 3 marked both values as the smallest and largest effects in the cited literature. Neither number could be traced to any source in that citation set. Both were figures with no citation behind them. This section records what the search for a source turned up, because what it found matters in its own right and is not just a bookkeeping fix.

Six of the eight cited papers report rates of return or point-spread cover rates, not win-probability calibration gaps. The two quantities cannot be swapped for each other without attaching the odds. A return R on a favorite priced at raw probability q corresponds to a calibration gap of the vig share plus q times R . At this sample's prices a -5.5 percent return corresponds to roughly -1.6 points, and a -23 percent return to roughly -16. Converted honestly, the classic return literature spans a far wider range than 1.7 to 5.1 and points the opposite way. That is because it concerns parimutuel horse racing rather than a two-outcome book.

Newall and Cortis (2021), the one literature review in the citation set and the obvious place such a range would live, reports only directions and never effect sizes. Levitt (2004) and Paul and Weinbach (2008) report point-spread cover rates against a 52.38 percent break-even, not moneyline calibration. Moskowitz (2021) explicitly excludes favorite-longshot bias from its scope. Angelini and De Angelis (2019) study three-outcome football markets. Griffith (1949) and Thaler and Ziemba (1988) concern parimutuel horse racing.

The moneyline branch rests largely on Woodland and Woodland (2001), whose conversion method was questioned by Berkowitz, Depken and Gandar (2018) on the grounds that it overstates bookmaker commission and understates underdog win probability. Those authors reported in 2011 that the effect holds for the first three seasons but disappears over the following seven, as the market becomes more efficient.

The marks were therefore removed rather than replaced with sourced ones, since no replacement in those units could be defended, and the power argument in the main paper is made against this sample's own break-even threshold instead. One coincidence is worth recording: 1.7 is numerically almost identical to this paper's own largest bucket deviation, +1.73 points in the .75 to .80 bucket.

S6. The flat-stake return intervals, checked against a simulated null

Losak and Kalamvokis (2025) make three claims about this literature. Tests of betting profitability are usually run as a t-test or z-test on mean profit per unit

staked. Per-bet profit is skewed, because a winning longshot pays several units while a loss always costs one. And under skew, the true rejection rate of such a test exceeds its nominal level. Their proposed fix is to simulate the null and read the interval off the empirical distribution.

The argument applies to section 5.5 of the main paper and not to section 5.1. The primary calibration test compares a win indicator with a probability, which is bounded. It is already evaluated against the exact Poisson-binomial distribution rather than a normal approximation, with a true size of .0484 reported in S2. The return analysis is the payout-weighted quantity their critique targets, and it does use the mean plus or minus 1.96 standard errors. It was therefore checked. Everything below is produced by `src/return_simulation.py`.

Bet group	n	Return	Normal 95% CI	Simulated 95% CI	Per-bet skew	True size
All favorites	15,351	-4.06%	-5.13 to -3.00	-5.12 to -3.00	-0.55	5.0%
All underdogs	15,351	-3.91%	-6.57 to -1.25	-6.66 to -1.21	+2.51	4.9%
Favorites above .70	6,840	-2.90%	-4.04 to -1.75	-4.04 to -1.73	-1.43	4.9%

Table S6. Simulated null against the normal approximation, 100,000 draws for the intervals and 40,000 for the size estimate, seed 20260818. No bound moves by more than 0.09 percentage points and the test holds its nominal .05 size, so the intervals reported in section 5.5 stand.

The intervals survive, but the mechanism they describe is real and it is present in these prices. Repeating the size calculation on subsamples, at the sample sizes this literature actually works with, reproduces their result. The test rejects too often, longshots are worse than favorites, and the problem fades as n grows.

Bet group	n	Per-bet SD	Skew of the mean	True size of the t-test
All favorites	100	66.4%	-0.03	5.4%
All favorites	1,000	67.6%	-0.01	5.0%
All underdogs	100	178.4%	+0.28	6.5%
All underdogs	250	169.3%	+0.16	5.7%
All underdogs	1,000	171.1%	+0.08	5.2%
All underdogs	5,000	171.6%	+0.05	5.0%
Favorites above .70	100	46.7%	-0.17	5.7%
Favorites above .70	1,000	48.5%	-0.05	5.2%

Table S7. The same two-sided test at nominal .05, run on subsamples of these prices. The simulated interval holds its size at 5.0 percent in every row. Reported whether or not it favors this paper: at $n = 100$ an underdog return test rejects a true null 6.5 percent of the time.

Two things follow. The return intervals in section 5.5 are safe at this sample size, and the reason is n rather than any property of the estimator. And consider a study of this kind run on one season of one bet type, which is common in the published record. It would be reading a test whose true size is closer to .065 than to .05, while reporting it as .05.

S4. Season-by-season results

Favorites above .70 vig-free implied probability, by season. No season rejects; the smallest p is .107. The Cochran Q heterogeneity statistic over these thirteen strata is 8.37 on twelve degrees of freedom, $p = .756$, with I-squared at 0 percent.

Season	n	Observed	Implied	Gap (pp)	z	p
2007-08	601	81.70%	81.00%	+0.70	0.44	.658
2008-09	565	82.48%	80.31%	+2.17	1.32	.188
2009-10	529	81.66%	80.28%	+1.38	0.81	.417
2010-11	536	82.46%	79.94%	+2.52	1.48	.140
2011-12	442	81.90%	80.00%	+1.90	1.01	.311
2012-13	525	80.76%	79.77%	+0.99	0.57	.567
2013-14	530	80.75%	80.43%	+0.33	0.19	.846
2014-15	585	80.68%	80.42%	+0.27	0.16	.869
2015-16	546	82.05%	81.51%	+0.54	0.33	.741
2016-17	490	77.96%	80.78%	-2.82	-1.61	.107
2017-18	513	78.75%	79.38%	-0.62	-0.35	.723
2018-19	539	81.08%	80.07%	+1.00	0.59	.555
2019-20	439	77.68%	79.02%	-1.34	-0.70	.484

Table S4. Exploratory, not pre-specified, and not corrected for multiplicity.

S5. Reproducing every number

Clone the repository, install the requirements, and run `bash run_all.sh`. The pipeline runs in nine steps and takes about five minutes. Data verification runs first and halts the pipeline on failure.

Step	Script	Produces
1	verify_data.py	full-sample cross-validation, schedule and outcome checks
2	build_dataset.py	exclusions, probabilities, favorite side, break-even
3	primary_analysis.py	primary test, calibration regression, buckets, returns
4	robustness.py	seasons, Cochran Q, extreme tail, season bootstrap
5	figures.py	Figures 1 and 2
6	normalization_robustness.py	five vig-removal rules (Table 5)
6	dependence_robustness.py	cluster-robust variance, team bootstrap (Table 4)
7	power_curve.py	Figure 3, Table 1, exact power (Tables S2 and S3)
8	shin_equivalence_proof.py	S1 verification (Table S1)
9	return_simulation.py	simulated null for the return intervals (Tables S6 and S7)

Table S5. Every statistic in the main paper is printed to the console and written to results/ as JSON or CSV.

A separate script, `src/fetch_source_data.py`, re-downloads the complete original files, prints their SHA-256 hashes, and confirms that every column kept in the local subsets matches the original row for row. The frozen analysis plan is at `preregistration/analysis_plan.md`, with a note on its history stating when it was typed into the repository and what evidence does and does not exist for the date it was frozen. A running record of every correction made to this paper, including the two documented in S2 and S3, is at `docs/errata.md`.